Reviewer Pack — Minimal Local Index in Tropical Geometry Bounds DPLL Path Le…

Entry 70dcad1bedab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 13:10:15 UTC

Minimal Local Index in Tropical Geometry Bounds DPLL Path Length

Entry ID: `70dcad1bedab` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 13:10:15 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): DPLL Search Trees in Complexity Theory

Statement:

> For every CNF φ with m clauses and n variables, the minimal local index in tropical geometry (mli(φ)) of its associated matroid is linearly correlated with the DPLL path length (dpll(φ)), such that mli(φ) = Θ(dpll(φ)).

Rationale (proposer's reasoning):

> Tropical geometry provides a geometric representation of Boolean functions, and the local index in tropical geometry measures the complexity of this representation. If this invariant is indeed related to the DPLL path length, it could suggest a deeper connection between geometric representations and algorithmic complexity.

Taxonomy category: `TROPICAL GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7f1e4bdb33fa03ad`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for 30 random seeds and instances of size n ≤ 40, the correlation coefficient between mli(φ) and dpll(φ) is ≥ 0.8 AND the mean absolute difference between mli(φ) and dpll(φ) is ≤ 3.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND DPLL path length` - `minimal local index IN tropical geometry AND complexity theory DPLL trees` - `CNF φ matroid AND linear correlation WITH DPLL path length`

Top relevant hits considered: - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1204.3875v2] Tropicalizing vs Compactifying the Torelli morphism - [http://arxiv.org/abs/1204.6154v2] Local Tropicalization - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1610.00298v4] Khovanskii bases, higher rank valuations and tropical geometry - [http://arxiv.org/abs/1810.06267v2] Small Space Stream Summary for Matroid Center - [http://arxiv.org/abs/1811.07464v1] Towards Nearly-linear Time Algorithms for Submodular Maximization with a Matroid Constraint - [http://arxiv.org/abs/2111.11378v2] The Young matroid: A multiset extension of the Catalan matroid to arbitrary Young diagrams

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        cnf = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll(cnf, assignment={}):
        literals = set()
        for clause in cnf:
            literals.update(clause)

        literal = next((l for l in literals if l not in assignment and -l not in assignment), None
        if literal is None:
            return True

        def propagate(lit):
```

```
            new_assignment = assignment.copy()
            new_assignment[lit] = True
            for clause in cnf:
                if lit in clause:
                    clause.remove(lit)
                elif -lit in clause:
                    clause.remove(-lit)
                if not clause:
                    return False
            return True

        def backtrack():
            while literal is not None:
                if propagate(literal):
                    result = dpll(cnf, assignment=new_assignment)
                    if result:
                        return True
                else:
                    del new_assignment[lit]
                    literal = next((l for l in literals if l not in new_assignment and -l not in n
            return False

        return backtrack()

    def solve(cnf):
        return dpll(cnf)

    n_values = [5, 10, 15, 20, 30, 40]
    mli_values = []
    dpll_path_lengths = []

    for n in n_values:
        for _ in range(5):  # Ensure at least 30 instances per seed
            cnf = generate_cnf(n, random.randint(1, n * (n - 1) // 2))
            mli_value = len(cnf)  # Simplified local index as number of clauses
            dpll_path_length = solve(cnf)

            mli_values.append(mli_value)
            dpll_path_lengths.append(dpll_path_length)

    correlation_coefficient = sum((x - mean_x) * (y - mean_y) for x, y in zip(mli_values, dpll_pat
    mean_mli = sum(mli_values) / len(mli_values)
    mean_dpll = sum(dpll_path_lengths) / len(dpll_path_lengths)

    if correlation_coefficient >= 0.8 and abs(mean_mli - mean_dpll) <= 3:
        conjecture_holds = True
        counterexample = ""
    else:
        conjecture_holds = False
        counterexample = "correlation_coefficient=<{}> mean_diff=<{}>".format(correlation_coeffici

    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(mli_values),
        "n_max": max(n_values),
```

```
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 2

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print("TRIAL: {}".format(result))
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_metric_value, std
    elif support_fraction >= 0.8:
        print("RESULT: SUPPORTED mean={} std={} support_fraction={}".format(mean_metric_value, std
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print("RESULT: FALSIFIED counterexample=\"{}\" first_failing_seed={}".format(results[resul
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59a3647f.py", line 104, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59a3647f.py", line 73, in run_trial
    dpll_path_length = solve(cnf)
                       ^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59a3647f.py", line 63, in solve
    return dpll(cnf)
           ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59a3647f.py", line 60, in dpll
    return backtrack()
           ^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_59a3647f.py", line 50, in backtrack
    while literal is not None:
          ^^^^^^^
UnboundLocalError: cannot access local variable 'literal' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support conditions could not be evaluated. | next: Debug the test code to ensure it runs successfully and produces the required data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23249
2	propose	ollama_remote	glm4:latest	0	0	14591
3	propose	ollama_remote	glm4:latest	0	0	12799
4	preregistration	ollama_remote	glm4:latest	0	0	15800
5	novelty	ollama_remote	glm4:latest	0	0	22000
6	novelty	ollama_remote	glm4:latest	0	0	16623
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18457
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16960
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8752
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20227
11	judge	ollama_remote	glm4:latest	0	0	9169

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 178627 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/70dcad1bedab.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/70dcad1bedab.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/70dcad1bedab.tar.gz` (if generated)